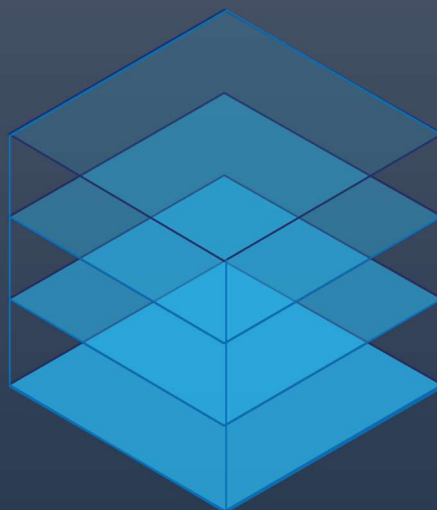




MISO

Maestría en Ingeniería de Software

PROYECTO - ENTREGABLE 4 DOCUMENTO DE ESCENARIOS Y RESULTADOS DE LAS PRUEBAS DE ESTRÉS

GRUPO 11

SERGIO FERNANDO BARRERA MOLANO
HAROLD ANDRÉS BARTOLO MOSCOSO
EDWIN HERNÁN HURTADO CRUZ
JUAN JOSÉ RESTREPO BONILLA

202517034
202513889
202326341
202516633

Escenarios y resultados de las pruebas de estrés

1. Objetivo general de las pruebas.

Evaluar el comportamiento y la capacidad de la aplicación web desplegada en AWS frente a un incremento progresivo de usuarios concurrentes, identificando el punto en el que el sistema comienza a degradar su rendimiento o falla.

2. Entorno de Pruebas.

Esta sección describe la infraestructura y las condiciones bajo las cuales se realizaron las pruebas. El entorno se diseñó para replicar el comportamiento de producción en un contexto controlado, empleando recursos y servicios reales de AWS para garantizar resultados representativos.

Se tiene los siguientes componentes de arquitectura en el despliegue de AWS.

Componente	Descripción
Región AWS	us-east-1
Balanceador de carga (ALB)	Distribuye peticiones HTTP hacia las instancias EC2 de la capa web
Instancias EC2 (Auto Scaling Group)	3 instancias t3.small para la capa web
Base de datos (RDS)	PostgreSQL (db.t3.small) utilizada para autenticación y almacenamiento persistente
Almacenamiento de objetos (S3)	Contiene los archivos asociados a los usuarios
Monitoreo (CloudWatch)	Recolección de métricas de CPU, red y respuesta del balanceador
Amazon SQS o Amazon Kinesis	Intercambio de mensajes de forma confiable, rápida y asíncrona.
Herramienta de prueba	Apache JMeter 5.6.3 ejecutado en Windows
Tipo de prueba	Estrés (simulación progresiva de múltiples usuarios intentando autenticarse simultáneamente)

3. Escenario de prueba ejecutado.

En esta sección se describe el diseño del escenario que se utilizó para someter la aplicación a condiciones de carga alta / extrema.

Se tuvieron 02 enfoques así:

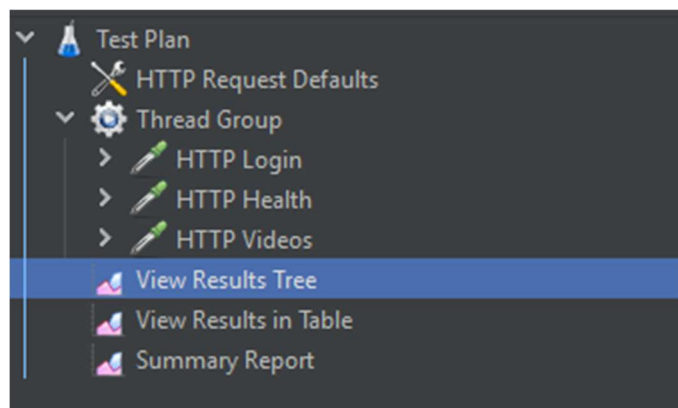
1. Realizar peticiones de Login (estrés a capa Web de Login), posterior con el bearer obtenido realizar la petición de estatus de la aplicación (estrés capa API) y finalizar realizar la petición de los videos de la aplicación (estrés del API) para estresar la parte de consultas hacia el backend de la aplicación.
2. Realizar el Login de forma manual con el FastAPI para obtención del bearer, y con este realizar la petición de estatus de la aplicación (estrés capa API) y finalizar realizar la petición de los videos de la aplicación (estrés del API) para estresar la parte de consultas hacia el backend de la aplicación.

Estos enfoques se realizaron dado que se evidencio previamente en la entrega pasada que el API de logeo y generación del token de autenticación por lo generar tiende a fallar antes que los API de la capa Web diseñados (funcionalidad del APP).

Parámetro	Valor
Objetivo	Evaluar el punto de falla mediante el aumento progresivo de peticiones al API del APP.
Endpoint probados	POST /auth/Login GET /health GET /api/videos
Usuarios concurrentes	Incremento progresivo de 1 a 2000 usuarios
Ramp-up period	De 1 a 10 segundos
Duración total de la prueba	2 minutos
Datos de prueba	Cuentas de usuario válidas creadas previamente

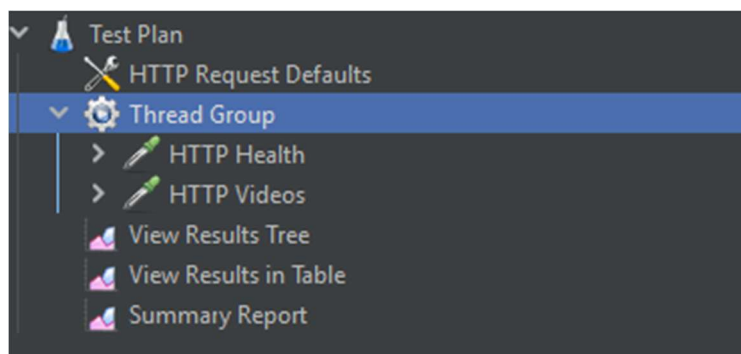
En la siguiente imagen se puede apreciar la configuración de Jmeter, donde se aprecia el Thread Group, el cual ejecuta las pruebas.

Enfoque 01: Estrés al API de autorización y generación de token.



En esta configuración cada “thread” o usuario realiza el Login para obtener el bearer y con ello realizar el consumo de las API de Health y Videos.

Enfoque 02: Estrés al API de /health y obtención de /videos



En esta configuración cada “thread” o usuario necesita a priori el bearer de autorización, el cual se genera de forma manual, con ellos realizar el consumo de las API de Health y Videos.

4. Resultados.

A continuación, se presentan los resultados obtenidos durante la ejecución de la prueba. Se incluyen las métricas de rendimiento, los errores detectados y el comportamiento general de la aplicación ante el aumento de carga.

Al iniciar el proceso de sobrecarga de forma gradual, observamos que el aplicativo responde correctamente, sin embargo, al intentar realizar un spike de 1000 usuarios en un tiempo rampa de 20 segundos (50 usuarios ingresando por segundo) al llegar a más de 112 usuarios concurrentes se chashea como se observa en las siguientes imágenes, evidenciamos que el elemento que falla inicialmente es el autorizador (Login para generar bearer).

Enter Logname (C:\Users\wang\Documents\MTIO-0209\Probes\EnterDates Logname Meter (3.8))

File Edit Search View Options Tools Help

00:02:20 0 / 1000

Test Plan

- HTTP Request Defaults
- Thread Group
 - HTTP Login
 - HTTP Health
 - HTTP Video
- View Results Tree
- View Results in Table
- Summary Report

View Results in Table

Name: View Results in Table

Comments:

Write results to file / Read from file

Filename: Browse Log Display Only Errors Success Configure

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Send Bytes	Latency	Connect Time(ms)
867	23:15:54.273	Thread Group 1-412	HTTP Health	205		196	247	205	83
868	23:15:54.272	Thread Group 1-421	HTTP Login	60306		262	294	60306	82
869	23:15:54.260	Thread Group 1-413	HTTP Health	186		196	247	186	84
870	23:15:54.264	Thread Group 1-422	HTTP Login	60184		262	294	60184	83
871	23:15:54.312	Thread Group 1-423	HTTP Login	60186		262	294	60186	83
872	23:15:54.314	Thread Group 1-424	HTTP Login	60161		262	294	60161	83
873	23:15:54.317	Thread Group 1-416	HTTP Health	186		196	247	186	84
874	23:15:54.337	Thread Group 1-415	HTTP Health	186		196	247	186	84
875	23:15:54.337	Thread Group 1-414	HTTP Health	186		196	247	186	84
876	23:15:54.354	Thread Group 1-425	HTTP Login	60163		262	294	60163	83
877	23:15:54.374	Thread Group 1-426	HTTP Login	60164		262	294	60164	83
878	23:15:54.384	Thread Group 1-427	HTTP Login	60164		262	294	60164	83
879	23:15:54.378	Thread Group 1-418	HTTP Health	185		196	247	185	84
880	23:15:54.362	Thread Group 1-417	HTTP Health	201		196	247	201	116
881	23:15:54.366	Thread Group 1-419	HTTP Health	189		196	247	189	83
882	23:15:54.619	Thread Group 1-420	HTTP Login	205		196	247	205	82
883	23:15:54.613	Thread Group 1-428	HTTP Login	60208		262	294	60208	81
884	23:15:54.653	Thread Group 1-430	HTTP Login	60188		262	294	60188	82
885	23:15:54.648	Thread Group 1-429	HTTP Login	60187		262	294	60187	82
886	23:15:54.673	Thread Group 1-431	HTTP Login	60186		262	294	60186	82
887	23:15:54.678	Thread Group 1-421	HTTP Health	186		196	247	186	84
888	23:15:54.679	Thread Group 1-423	HTTP Health	186		196	247	186	84
889	23:15:54.679	Thread Group 1-422	HTTP Health	186		196	247	186	84
890	23:15:54.695	Thread Group 1-432	HTTP Login	60164		262	294	60164	82
891	23:15:54.715	Thread Group 1-433	HTTP Login	60165		262	294	60165	83
892	23:15:54.646	Thread Group 1-434	HTTP Login	60164		262	294	60164	83
893	23:15:54.696	Thread Group 1-434	HTTP Health	212		196	247	212	121
894	23:15:54.717	Thread Group 1-425	HTTP Health	192		196	247	192	104
895	23:15:54.728	Thread Group 1-428	HTTP Health	172		196	247	172	54
896	23:15:54.552	Thread Group 1-435	HTTP Login	60211		262	294	60211	83
897	23:15:54.556	Thread Group 1-427	HTTP Health	205		196	247	205	82
898	23:15:54.672	Thread Group 1-406	HTTP Login	60179		262	294	60179	84
899	23:15:54.569	Thread Group 1-407	HTTP Login	60170		262	294	60170	84
900	23:15:54.613	Thread Group 1-438	HTTP Login	60164		262	294	60164	83
901	23:15:54.621	Thread Group 1-439	HTTP Health	186		196	247	186	84

☐ Scroll automatically ☐ Child sample? No of Samples: 900 Last Sample: 9081 Average: 1961 Success: 98%

JMeter en punto de quiebre, estrés medio alto

The screenshot shows the Burp Suite application window. The top menu bar includes File, Edit, Search, Run, Options, JUnit, Help, and a toolbar with various icons. The left sidebar contains a 'Test Plan' tree with nodes for HTTP Request Defaults, HTTP Login, HTTP Health, HTTP Status, View Results Tree (selected), View Results in Table, and Summary Report. The main area is divided into two panes. The top pane is 'View Results Tree' (VRT) with a 'Name' field set to 'View Results Tree'. Below it is a 'Comments' section. The bottom pane is 'View Results in Table' (VRI) with a 'Name' field set to 'View Results in Table'. The VRI pane shows a list of HTTP requests and responses. The selected item is an HTTP request with a status of 200. The response body is displayed in the VRT pane, showing a JSON message: '[\"detalle\": \"Error interno del servidor\"]'.

JMeter cuando la aplicación está completamente caída

Table 1 Análisis de los datos obtenidos de JMeter

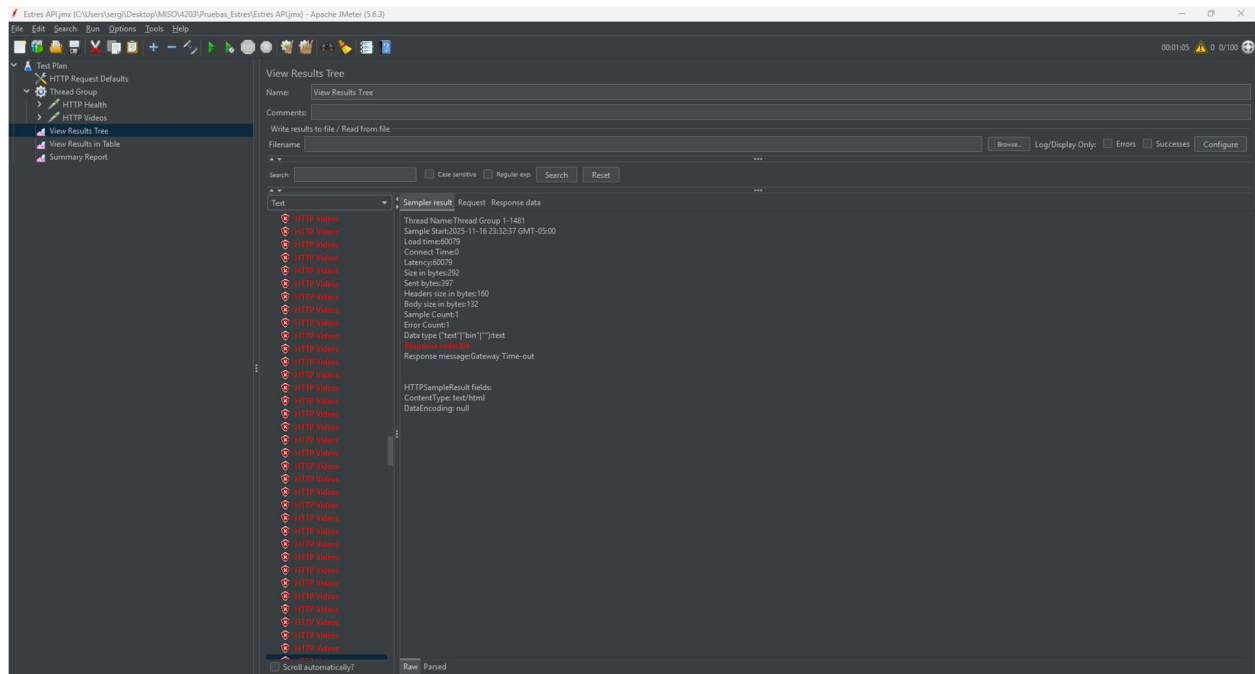
Petición	# Muestras	Promedio (ms)	Error %	Throughput (req/seg)	Observaciones
HTTP Login	1000	56.062	93.33%	12.5	El tiempo promedio de respuesta fue significativamente alto, evidenciando saturación en el proceso de autenticación. Más del 90% de las peticiones fallaron.
HTTP Health	1000	162	0.00%	12.6	Este endpoint, de baja complejidad, se mantuvo estable y rápido durante toda la prueba, lo que sugiere que el entorno general y el balanceador de carga estaban funcionando correctamente.
HTTP Videos	1000	59.833	93.33%	7.2	Se presentaron tiempos de respuesta altos y una tasa de error superior al 90%. Esto indica que las peticiones a endpoints que requieren autenticación y acceso a datos presentan serios problemas de concurrencia dado el endpoint de Login.

Estos valores reflejan que la aplicación no mantiene estabilidad bajo carga moderada, y que el componente de autenticación (/auth/login) es el más sensible al estrés.

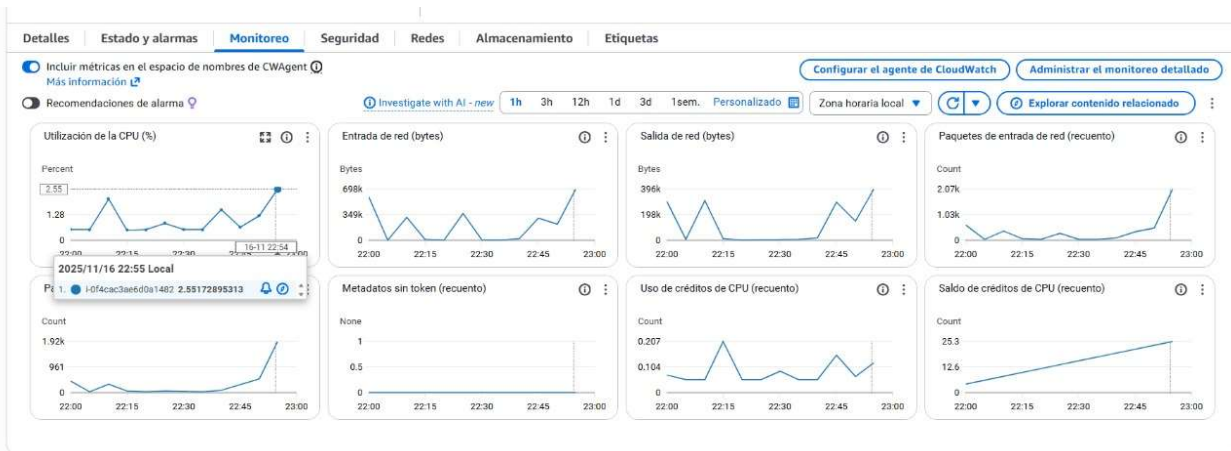
Por este motivo se implementó el enfoque 02, en el cual se tiene una autenticación previa (genera de forma manual en el WEB de fastAPI), y se realiza la petición y consumo de las API de /health y /videos que son propias del aplicativo, de esta forma buscar generar el estrés en las colas de mensajes y las peticiones al backend para consulta de los videos que almacena la aplicación.

Bajo este nuevo enfoque se puede obtener un mejor estimado de fallo de la aplicación, mediante la metodología que aplica aumentos de forma progresiva a los usuarios que realizan peticiones, donde se identifica una capacidad de 1905 usuarios concurrentes generando peticiones al API de /videos, posterior a este número el aplicativo presenta fallos y el servicio se convierte en inestable o completamente caído.

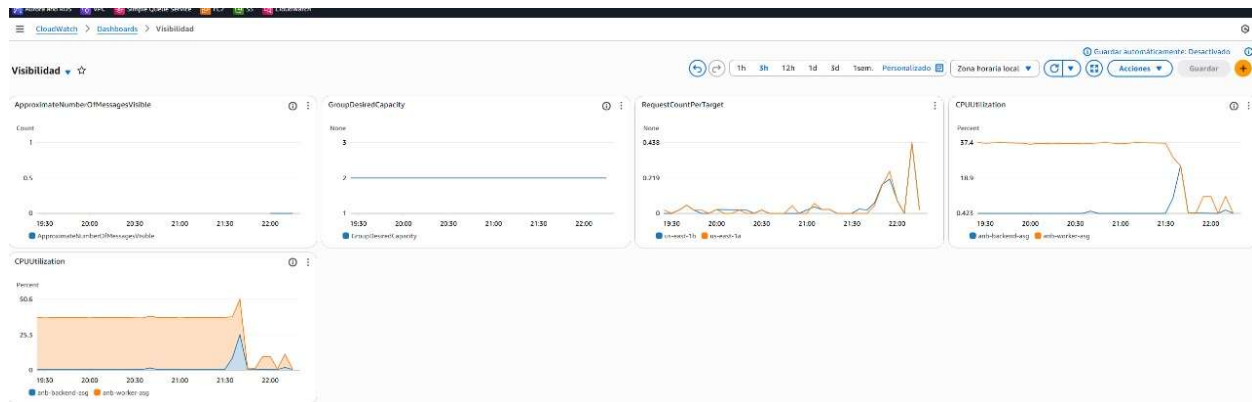
A continuación, se presentan las imágenes obtenidas de CloudWatch y de JMeter mientras se ejecutaban las pruebas de estrés bajo este nuevo enfoque:



En la imagen anterior podemos observar que al incrementar los usuarios concurrentes, llega el punto que el API fracasa y responde con error 504 “Gateway Time-Out”, evidenciando la caída del aplicativo, en este caso se evidencio que el aplicativo cae a intentar 1905 usuarios concurrentes.



En la imagen anterior podemos observar mediante el uso de CloudWatch en AWS, que se presenta un incremento en las peticiones (En el orden de los 2000) así mismo se incrementa el uso de la CPU que se utiliza para procesar este número de peticiones que se le están enviando al aplicativo.



En la imagen anterior podemos observar como el aplicativo llega a tener consumos de más del 30% en la CPU, solo intentando responder a las peticiones que se envían, posterior a que se llega al punto de quiebre y el aplicativo deja de enviar peticiones se realiza la recuperación a IDLE con bajo consumo.

5. Conclusiones.

La aplicación presenta limitaciones de concurrencia: El sistema mantiene estabilidad hasta aproximadamente 100-110 usuarios simultáneos; a partir de ese punto, los tiempos de respuesta superan los 20 segundos y aumentan los errores HTTP 500 y 504, evidenciando saturación en el backend.

El proceso de autenticación es el principal cuello de botella: El endpoint `/auth/login` concentra el mayor tiempo de respuesta y más del 90% de errores, lo que sugiere sobrecarga en la gestión de generación de tokens de autenticación.

Al realizar las pruebas a los API de `/health` y `/videos` con un usuario previamente logeado se puede evitar caer en el cuello de botella evidenciado previamente del endpoint de `/Login`, permitiendo hasta 1905 usuarios de forma concurrente antes de fallar.

6. Análisis Técnico de la Falla.

Cuello de botella en autenticación: El endpoint `/auth/login` muestra el mayor tiempo de respuesta y un tercio de los errores. Esto sugiere saturación en la gestión de sesiones o consultas al RDS para validar el logeo y la generación del token de autenticación.

Diferencia entre endpoints: El endpoint `/health` responde correctamente, lo que demuestra que la infraestructura y el balanceador funcionan. El problema radica en los servicios que acceden a datos (login), ya que, si se cae el Login no se podría acceder a la aplicación ya que no pasaría la validación de token para acceder a los demás APIs. Sin embargo, cuando se tiene un token válido, se permiten hasta 1905 usuarios concurrentes, y se evidencia el uso de la CPU en las máquinas AWS y la respuesta del sistema.